

unique_ptr — Խորը ուղեցույց

C++ Smart Pointer presentation

Այս ուղեցույցը մանրամասն բացատրում է `std::unique_ptr`-ը՝ ինչ է, ինչպես է աշխատում, ownership, move semantics, `make_unique`, custom deleter, օրինակներով և հարցազրույցի հարցերով:

1. Ի՞նչ է unique_ptr-ը

`std::unique_ptr`-ը smart pointer է, որ ունի **exclusive ownership** resource-ի վրա: Մեկ resource-ը կարող է պատկանել **միայն մեկ** `unique_ptr`-ի: Scope-ից դուրս գալիս՝ ավտոմատ `delete`:

```
#include <memory>

std::unique_ptr<int> p = std::make_unique<int>(10);
cout << *p;           // 10
// delete պետք չէ -- scope-ի վերջում ավտոմատ
```

RAII-ի լավագույն օրինակ՝ heap հիշողությունը կառավարվում է ավտոմատ, չկա leak:

2. Exclusive Ownership — copy ՉԿԱ

```
std::unique_ptr<int> a = std::make_unique<int>(5);
std::unique_ptr<int> b = a;           // COMPILE ERROR -- copy չկա
std::unique_ptr<int> c = std::move(a); // OK -- ownership փոխանցվում
// հիմա a == nullptr, c-ն ունի resource-ը
```

`unique_ptr`-ը **չի copy լինում** (միայն մեկ owner): Ownership-ը փոխանցելու՝ `std::move` -> հին pointer-ը դառնում `nullptr`:

3. make_unique (նախընտրելի)

```
auto p = std::make_unique<int>(10);           // int
auto arr = std::make_unique<int[]>(100);       // array
auto obj = std::make_unique<MyClass>(a, b);    // object ctor args-ով
```

Ինչու make_unique (ոչ `unique_ptr<int>(new int)`). - Ավելի կարճ, exception-safe - Չկա բացահայտ `new`

4. Կարևոր մեթոդներ

```
auto p = std::make_unique<int>(10);

*p;           // dereference
p.get();      // raw pointer (ownership չի փոխանցում)
p.reset();    // ազատում resource-ը (delete), p = nullptr
p.reset(new int(20)); // հին ազատում, նոր վերցնում
p.release();  // ownership հանում, վերադարձնում raw pointer (delete Չի անում)
if (p) { }    // ստուգում` null չէ
```

5. Ownership transfer (Ֆունկցիաների հետ)

```
std::unique_ptr<int> create() {
    return std::make_unique<int>(42); // ownership վերադարձնում (move ավտոմատ)
}

void consume(std::unique_ptr<int> p) { // ownership վերցնում
    cout << *p;
} // p ոչնչանում -> delete

auto p = create();
consume(std::move(p)); // ownership փոխանցում -> հիմա p == nullptr
```

6. Custom Deleter

```
auto fileDeleter = [](FILE* f) { if (f) fclose(f); };
std::unique_ptr<FILE, decltype(fileDeleter)> file(fopen("data.txt", "r"), fileDeleter);
// scope-ի վերջում -> fclose ավտոմատ
```

Custom deleter-ով կարաս կառավարել ոչ-memory resource (file, socket, handle):

7. Ինչու unique_ptr

- Zero overhead (raw pointer-ի չափով, արագ)
- Exclusive ownership -> պարզ ownership-ի մոդել
- Ավտոմատ delete -> չկա leak
- Move-only -> չկա պատահական copy
- DEFAULT smart pointer (եթե shared պետք չէ)

8. unique_ptr vs raw pointer

```
int* raw = new int(5); // ձեռքով delete պետք, leak-ի վտանգ
// vs
auto smart = std::make_unique<int>(5); // ավտոմատ, safe
```

9. Հարցազրույցի հարցեր և պատասխաններ

unique_ptr ինչ է:

Smart pointer exclusive ownership-ով; մեկ resource -> մեկ owner; scope-ի վերջում ավտոմատ delete:

Ինչո՞ւ copy չկա:

Exclusive ownership -- երկու owner -> double delete. Միայն move (ownership transfer):

make_unique vs unique_ptr(new):

make_unique ավելի կարճ, exception-safe, չկա բացահայտ new:

Ownership ինչպես էս փոխանցում:

std::move -> հին pointer nullptr դառնում:

get() vs release():

get() raw pointer տալիս (ownership պահում); release() ownership հանում, raw տալիս (delete չի անում):

reset() ինչ է անում:

Ազատում է ներկայիս resource-ը (delete), optional նորը վերցնում:

Overhead:

Գրեթե zero -- raw pointer-ի չափով, արագ:

Ամփոփում (Cheat Sheet)

`unique_ptr` = EXCLUSIVE ownership (մեկ owner), ավտոմատ delete, zero overhead

`make_unique<T>(args)` -> ստեղծում (նախընտրելի)

`*p / p->` -> dereference

`p.get()` -> raw pointer (ownership պահում)

`p.release()` -> ownership հանում (delete Չի)

`p.reset()` -> ազատում (delete)

`std::move(p)` -> ownership փոխանցում (`p -> nullptr`)

COPY ՉԿԱ (միայն move) | move-only

DEFAULT smart pointer (shared պետք չլինելու դեպքում)